NRLC-01: New Relic vs. Dynatrace — Platform Comparison

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 1 of 9 | **Created:** April 2026 | **Last Updated:** 10/31/2026

Overview

This deep dive establishes the conceptual foundation for migrating from New Relic to Dynatrace. Before any tooling, queries, or transformations, a clear understanding of how the two platforms differ in **architecture, ingest model, query language, entity hierarchy, IAM, and pricing** prevents the most common migration mistakes — those that come from assuming the platforms are functionally equivalent.

Use this notebook as the conceptual reference; the procedural sequence for executing a migration lives in the companion **NR2DT** series.

Sprint 1.337 (April 2026) Updates

Three sprint-1.337 changes affect the NR→DT component conversion patterns:

1. **OTel `service.name` enrichment** — Dynatrace now uses the OpenTelemetry `service.name` resource attribute to enrich the existing service entity (Smartscape displays `service.name (detected name)`; new `dt.service.name` field). NR services that emit OTel via the New Relic Collector → Dynatrace path map cleanly without two-services-per-process duplication.
2. **OneAgent primary fields/tags at the source** (`dt.security_context`, `dt.cost.costcenter`, `dt.cost.product` + customer-defined primary tags) — top-level on all signals. Reduces parse-processor work for migrated NR data.
3. **Entity API: `attributes` removed + Event API: `metadata` removed from `GET /events`** (sprint-337 Dynatrace API). Update any conversion-engine code (Dynatrace-NewRelic / nrql-engine / nrql-translator) that parses these properties — the engine repositories are listed in **NRLC-09 §1**.

Table of Contents

1. Architectural Models
2. Data Ingest & Storage
3. Query Languages: NRQL vs. DQL
4. Entity Hierarchy
5. Identity, Access & Multi-Tenancy
6. Pricing Models
7. Concept Mapping Reference

Prerequisites

Requirement	Details
Audience	Architects, platform engineers, SRE leads evaluating or executing a NR→Dynatrace migration
Familiarity	Working knowledge of one platform; this notebook bridges to the other
Companion	NR2DT-01 — Discover (procedural counterpart)

Engine Support (Phase 11–24)

The open-source `Dynatrace-NewRelic` engine covers the cross-platform concepts this notebook introduces. Key transformers you'll hear about again in later notebooks:

Topic	Transformer / Subcommand	Phase
APM agent swap per language (Java / .NET / Node / Python / Ruby / PHP / Go)	`agents/` + `migrate.py agents --language <lang>`	16

Topic	Transformer / Subcommand	Phase
`newrelic.*()` SDK call scanner	`custom_instrumentation_translator` + `migrate.py scan-instrumentation`	16
Browser RUM app config	`browser_rum_transformer`	16
Mobile RUM app (8 platforms)	`mobile_rum_transformer`	16
Users / Teams / Roles / SAML / SCIM	`identity_transformer`	17
Cloud (AWS 16 / Azure 8 / GCP 8)	`cloud_integration_transformer`	18
Kubernetes → DynaKube	`kubernetes_transformer`	18
Tenant Gen3 API probe	`migrate.py preflight`	14

See **NRLC-09** for the full 42-transformer + 14-subcommand inventory and the coverage matrix (migration utilities GitHub) for the end-to-end NR-surface → DT-equivalent map.

1. Architectural Models

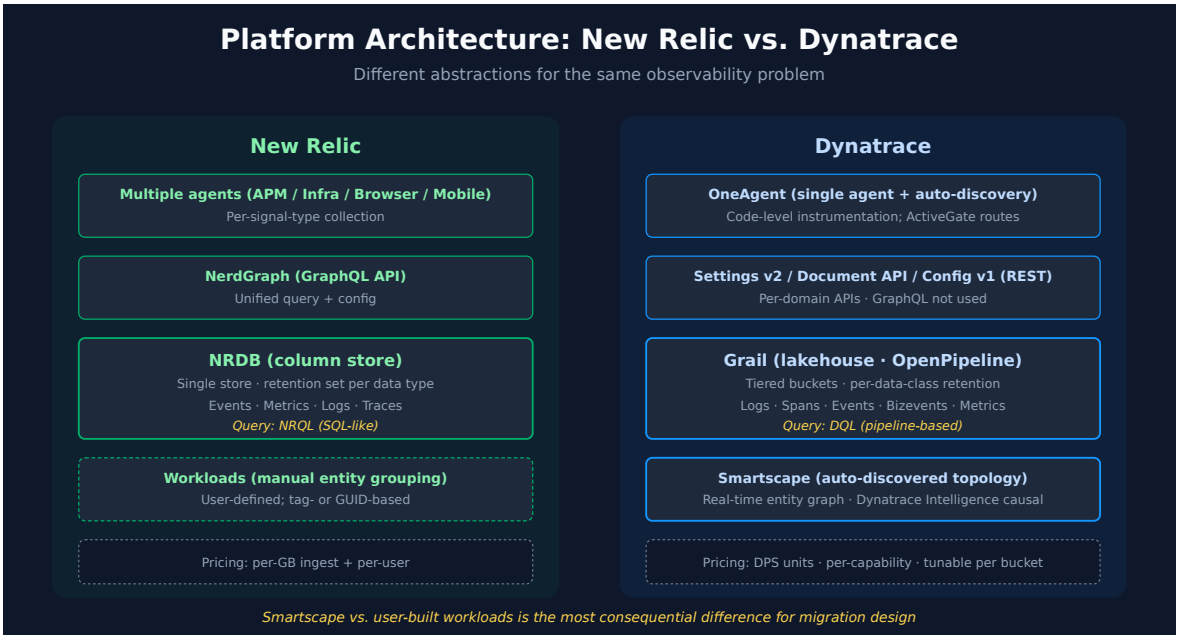
New Relic is built around a unified telemetry data platform (NRDB) where every signal — metrics, events, logs, traces — is stored as discrete events in a columnar database. Agents (APM, Infrastructure, Browser, Mobile, OpenTelemetry collector) push data via HTTP. NerdGraph (GraphQL) is the unified API for both data queries and configuration.

Dynatrace is built around **Smartscape topology + Dynatrace Intelligence + Grail data lakehouse**. OneAgent does deep instrumentation including code-level discovery; ActiveGate handles routing. Configuration lives in the Settings (v2) and Document (`/platform/document/v1`) APIs; data lives in Grail (queried via DQL). The platform infers entity relationships automatically (Smartscape) and applies causal AI to detect root cause.

Aspect	New Relic	Dynatrace
Storage backend	NRDB (column store)	Grail (lakehouse: logs, spans, events, bizevents, metrics)
Agent model	Multiple agents per signal type	Single OneAgent (auto-discovery) + ActiveGate routing

Aspect	New Relic	Dynatrace
Topology	User-built workloads, manual entity grouping	Smartscape (auto-discovered, real-time)
AI model	NRDB anomaly detection + APM Proactive Detection	Dynatrace Intelligence causal engine + adaptive baselines
Configuration API	NerdGraph (GraphQL)	Settings v2, Document API, Config v1 (REST)
Query language	NRQL (SQL-like)	DQL (pipeline-based)

The most consequential difference: **Dynatrace infers entity relationships automatically; New Relic relies on user-defined workloads.** This affects how dashboards, alerts, and SLOs scope their target entities.



2. Data Ingest & Storage

New Relic

- **Single ingest endpoint per data type** (Metric, Event, Log, Trace APIs)
- All data lands in NRDB; retention is set per data type (8 days for APM data, for example)

- Drop rules and parsing rules apply at ingest
- **Data Plus** adds *"up to an additional 90 days of retention (for most data types)"*

Dynatrace

- **OneAgent + ActiveGate + OpenPipeline + Grail** — multiple ingest paths
- Data routed to **named buckets** with tiered retention (14d / 30d / 365d depending on use case)
- OpenPipeline performs parse, mask, enrich, route, and metric extraction *before* storage
- Per-bucket pricing: "Retain with Included Queries" or "Usage-based"

Migration implication: New Relic's per-data-type retention maps poorly to Dynatrace's per-bucket retention. Plan retention per data class during migration design (see **ORGNZ-02 — Understanding Grail Buckets** and **ORGNZ-99 — Best Practice Summary** for the recommended bucket strategy template).

> **Sources:** Manage data retention (New Relic docs).

3. Query Languages: NRQL vs. DQL

NRQL is SQL-like with `SELECT ... FROM ... WHERE ... FACET ... TIMESERIES`. Single statement, declarative.

DQL is pipeline-based: `fetch <source> | filter ... | summarize ... | sort ... | limit N`. Each `|` is a stage that transforms the previous stage's output.

Side-by-Side Translation

NRQL:

```
SELECT count(*), average(duration) FROM Transaction
WHERE appName = 'checkout-service' SINCE 1 hour ago
FACET host.hostname TIMESERIES 5 minutes
```

Equivalent DQL:

```
fetch spans, from:-1h
| filter dt.service.name == "checkout-service" and isNotNull(endpoint.name)
| makeTimeseries {count = count(), avg_dur = avg(duration)}, interval:5m, by:
{host.name}
```

Key conceptual differences:
- **Source resolution:** NRQL's `FROM Transaction` is a NRDB event class — one record per request into a service. DQL's `fetch spans` is a Grail data object holding *every* span, including database calls, outbound calls and internal spans, so the translation needs `filter isNotNull(endpoint.name)`: the semantic dictionary describes `endpoint.name` (`stable`) as *"exclusively detected on request root spans"*, so the filter leaves one record per request. Without it, the count read about 2.8× the request count on the validation tenant (10/05/2026).
- **Service name:** map NR `appName` to `dt.service.name`, the Dynatrace service name. `service.name` is the OpenTelemetry resource attribute and is set only on spans that carry it, so filtering on it silently drops traffic that does not: on the validation tenant fewer than one span in five carried `service.name`, while every span carried `dt.service.name` (10/05/2026). Use `service.name` only for OpenTelemetry-only services.
- **Units:** NR `duration` is a number of seconds; DT `duration` is a duration value. Compare it with a duration literal (`duration > 500ms`) and divide by a unit to get a number (`avg(duration) / 1ms`).
- **Time:** NRQL uses `SINCE` / `UNTIL`; DQL uses `from:` / `to:` parameters on `fetch` / `timeseries`.
- **Facets:** NRQL `FACET x` becomes DQL `by:{x}` inside `summarize` or `makeTimeseries`.
- **Timeseries:** NRQL `TIMESERIES N minutes` becomes DQL `makeTimeseries ..., interval:Nm`.

See **NRLC-02: NRQL→DQL Translation** for the full translation surface (292 patterns), confidence scoring, and the open-source `nrql-engine` compiler.

4. Entity Hierarchy

New Relic

Entities are typed objects (APM Application, Host, Synthetic Monitor, Browser App, Mobile App, etc.) identified by **GUID**. Relationships are explicit but limited — often inferred from APM instrumentation. Workloads group entities manually.

Dynatrace

Smartscape automatically discovers and connects entities:

Process → Process Group → Host → Host Group

Plus: Service → Service Group, Application → Application Group, custom entities, Kubernetes entities (Pod, Cluster, Namespace, Workload).

New Relic Entity	Dynatrace Equivalent	Notes
APM Application	Service	Auto-detected by OneAgent code instrumentation
Host	Host	Auto-discovered; assigned to host groups
Container	Container Group	K8s container / pod-aware
Workload	OpenPipeline enrichment + IAM policy condition on enriched attribute	Manual definition; rules-based selectors
Synthetic Monitor	HTTP/Browser Monitor	Direct mapping for ping; scripts need adaptation
Browser App	RUM Application	OneAgent JS injection or manual tag
Mobile App	Mobile Application	Native SDK
Lambda Function	AWS Lambda	Cloud integration

Migration implication: NR's GUID references in dashboards and alert conditions don't map 1:1 to DT entity IDs. Use **entity selectors** (`type(SERVICE),entityName("checkout")`) for portability.

5. Identity, Access & Multi-Tenancy

New Relic

- **Account / Sub-account** structure with role-based permissions per account

- Standard roles include All product admin, Standard user and Read only
- Permissions per category (Alerts, APM, Infrastructure, Insights/Dashboards)

Dynatrace

- **Tenant** (single environment) with **Groups** ↔ **Policies** ↔ **Permissions** model
- IAM policies use ALLOW-WHERE statements with conditions on `storage:bucket-name`, `storage:table`, and DQL attribute filters
- **Bucket routing** + **OpenPipeline enrichment** define the data boundaries; IAM scopes access to those boundaries
- **Policy boundaries** (DT Gen3) constrain what conditions a policy can include

NR Concept	DT Equivalent	Mapping
Sub-account isolation	Tenant + OpenPipeline-enriched data + IAM policy bucket/attribute condition	Soft isolation via data-level attribute scoping
Read only role	Policy with bucket scope + DQL-attribute conditions	Fine-grained per-resource (Gen3 pattern)
All product admin role	Tenant admin policy	Full tenant access

Migration implication: A NR multi-account setup typically becomes a single DT tenant + an attribute taxonomy applied via OpenPipeline enrichment + bucket-scoped IAM policies. See **FAQ-01 — Host Group Naming Strategy**, **ORGNZ-02 / ORGNZ-99** (Grail bucket strategy), and the **IAM** series (especially IAM-04 / IAM-05 / IAM-11 WORKSHOP) for the design patterns.

> **Sources:** User management concepts (New Relic docs) — *"Standard roles include: All product admin"*.

6. Pricing Models

New Relic

- **Usage-based**: per GB of ingested data + per user (Full platform and Core are billed, Basic is not); a Data + Core Compute option is in Preview
- Single rate across all data types
- No retention tier discount

Dynatrace

- **Dynatrace Platform Subscription (DPS)**: consumption units (DPS) tied to specific capability use
- Different DPS rates per capability: log ingest, log retention with included queries, log usage-based queries, full-stack monitoring (GiB-hours), etc.
- **Bucket pricing tiers** allow optimization: keep hot data with included queries, cold data on usage-based

Cost Driver	New Relic	Dynatrace
Log ingest	$/GB	DPS per GB ingested
Log retention	Bundled in retention period	Separate DPS rate; tunable per bucket
Log queries	Free within retention	DPS per GiB scanned (usage-based buckets) or included (RWIQ)
Hosts	Per user (full-stack indirectly)	DPS per GiB-hour of host memory (FSM)
Synthetics	Per check	DPS per synthetic action (browser) or synthetic request (HTTP)

Migration implication: NR's flat rate often appears cheaper at small scale but doesn't scale efficiently. DT's tiered model lets you right-size cost per data class. A 30–60% reduction is typical when an OpenPipeline + tiered bucket strategy replaces NR's flat ingest. See **NRLC-07 Logs/Tags/Drops** for cost-optimization patterns.

> **Sources:**
>
> - Full-Stack Monitoring (DT docs) — *"Consumption is based on host memory"*
> - Log query (DT docs) — *"It is calculated per gibibyte scanned"*
> - Browser monitor (DT docs) — *"usage based on the number of synthetic actions executed"*
> - HTTP monitor (DT docs) — *"HTTP Monitor consumption is measured in synthetic requests"*
> - Pricing and billing (New Relic docs)

7. Concept Mapping Reference

Quick-reference table for every major concept this series touches:

New Relic	Dynatrace	Deep Dive
NRQL	DQL	NRLC-02
Dashboard (multi-page)	Document (single-page × N)	NRLC-03
Alert Policy	Workflow (Gen3)	NRLC-04
NRQL Alert Condition	Anomaly detector	NRLC-04
APM Condition	Adaptive Baseline (Dynatrace Intelligence)	NRLC-04
Synthetic Ping	HTTP Monitor	NRLC-05
Synthetic Browser	Browser Monitor	NRLC-05
Synthetic API	HTTP Monitor (multi-step)	NRLC-05
Synthetic Step Monitor	Browser Monitor (clickpath)	NRLC-05
SLO	SLO (Service-Level Objectives app)	NRLC-06
Workload	OpenPipeline enrichment + IAM policy condition on enriched attribute	NRLC-06
Tag	Tag (entity / Grail dimension)	NRLC-07
Drop Rule	OpenPipeline `Drop record` processor	NRLC-07

New Relic	Dynatrace	Deep Dive
Notification Channel (Email/Slack/PD)	Workflow task (connector or HTTP Request)	NRLC-04
Log Parsing Rule	OpenPipeline DPL parser	NRLC-07
Sub-account	Bucket + OpenPipeline enrichment + IAM	NRLC-01 §5
User Role	IAM Policy + Group	NRLC-01 §5

Summary

The two platforms solve the same observability problem with very different abstractions. Migration succeeds when each NR concept is mapped to its closest DT equivalent and remaining gaps are closed with intentional design — not by trying to make DT behave like NR.

Continue to **NR2DT-01 Discover** for the procedural wave plan + discovery runbook, or jump to a specific topic deep dive via the table above.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.